

1. Arquitectura y alimentación del Arduino Uno

Describe el microcontrolador que utiliza la placa Arduino Uno e indica cuántos pines digitales y cuántos pines analógicos posee. Menciona explícitamente los números de los pines que soportan salida PWM. Además, explica las distintas formas en que se puede suministrar energía eléctrica a la placa y señala el rango de voltaje recomendado cuando se emplea el conector de alimentación externa (jack de barril).

2. Ciclo de vida del programa y control de entradas/salidas digitales

Explica el propósito de las funciones `setup()` y `loop()` en un sketch de Arduino. Dentro de ese contexto, describe cómo se configura un pin para que actúe como salida digital y mediante qué instrucción se puede enviar un valor HIGH o LOW a dicho pin. Asimismo, indica cómo se lee el estado de un pulsador conectado a una entrada digital y qué función se utiliza para introducir una pausa de exactamente un segundo en la ejecución del programa.

3. Comunicación serie

Detalla el procedimiento para inicializar la comunicación serie en Arduino, especificando la función que se emplea y cómo se establece una velocidad de 9600 baudios. Compara las funciones que permiten enviar datos por el puerto serie: una sin salto de línea y otra que añade retorno de carro y salto de línea. Finalmente, explica cómo se comprueba si hay datos disponibles para leer en el búfer de recepción y qué función se usa para leer un único byte del puerto.

4. Modulación por ancho de pulso (PWM)

Define el significado de las siglas PWM (en español) y explica la utilidad de esta técnica en las salidas de Arduino. Indica la función específica que genera una señal PWM, el rango de valores que acepta para controlar el ciclo de trabajo y qué voltaje promedio se obtiene aproximadamente en un pin de 5 V cuando se le asigna un valor de 127. Complementa la respuesta enumerando todos los pines del Arduino Uno en los que está disponible la salida PWM.

5. Gestión del tiempo y generación de números aleatorios

Describe qué función devuelve el número de milisegundos transcurridos desde que el programa comenzó a ejecutarse. Razona por qué se desaconseja el uso de `delay()` en aplicaciones que deben leer sensores de manera continua o responder rápidamente a eventos. Por último, explica cómo se generan números pseudoaleatorios en Arduino, indicando tanto la función que los produce como el método recomendado para inicializar la semilla del generador a partir de una señal impredecible (por ejemplo, la lectura de un pin analógico sin conectar).

6. Código de desvanecimiento con for

Analiza el siguiente sketch y explica detalladamente qué hace, qué sucede con el LED, qué función controla la intensidad luminosa, por qué se utilizan dos bucles for y qué pines del Arduino Uno podrían emplearse para este propósito.

```
int led = 9;
void setup() {
  pinMode(led, OUTPUT);
}
void loop() {
  for (int brillo = 0; brillo <= 255; brillo++) {
    analogWrite(led, brillo);
    delay(10);
  }
  for (int brillo = 255; brillo >= 0; brillo--) {
    analogWrite(led, brillo);
    delay(10);
  }
}
```

7. Código con pulsador, LED y monitor serie

Explica el comportamiento completo del siguiente programa. Menciona cómo se configura y lee el botón, cómo se controla el LED en función del estado del pulsador, qué se envía por el puerto serie y qué función añade un salto de línea a los mensajes enviados.

```
int boton = 2;
int led = 13;
void setup() {
  pinMode(boton, INPUT);
  pinMode(led, OUTPUT);
}
```

```

    Serial.begin(9600);
}
void loop() {
    int estado = digitalRead(boton);
    if (estado == HIGH) {
        digitalWrite(led, HIGH);
        Serial.println("Pulsado");
    } else {
        digitalWrite(led, LOW);
        Serial.println("No pulsado");
    }
    delay(200);
}

```

8. Código con números aleatorios y PWM

Describe el funcionamiento de este sketch. En tu respuesta aclara: ¿por qué se utiliza randomSeed(analogRead(0))? ¿Qué rango de valores puede tomar la variable valor y cómo se refleja en la salida PWM? ¿Qué función envía texto al monitor serie sin salto de línea y cuál lo añade?

```

int ledPWM = 6;
void setup() {
    pinMode(ledPWM, OUTPUT);
    Serial.begin(9600);
    randomSeed(analogRead(0));
}
void loop() {
    int valor = random(0, 256);
    analogWrite(ledPWM, valor);
    Serial.print("Valor PWM: ");
    Serial.println(valor);
    delay(1000);
}

```

9. Error con PWM

El siguiente código pretende atenuar un LED, pero no funciona correctamente. Identifica el error, justifica por qué falla basándote en las características de los pines del Arduino Uno y escribe la(s) línea(s) corregida(s).

```

int led = 13;
void setup() {
    pinMode(led, OUTPUT);
}
void loop() {
    analogWrite(led, 128);
    delay(500);
}

```

10. Error de lectura analógica fuera de función

Encuentra el error principal del programa y proporciona la versión corregida. Explica por qué la variable valorSensor no se actualiza durante la ejecución y cómo debe resolverse para que el brillo del LED refleje continuamente el valor leído en el pin A0.

```

int valorSensor = analogRead(A0);
void setup() {
    pinMode(9, OUTPUT);
    Serial.begin(9600);
}
void loop() {
    int brillo = valorSensor / 4;
    analogWrite(9, brillo);
    Serial.println(valorSensor);
    delay(100);
}

```